KUMARAGURU COLLEGE OF TECHNOLOGY

COIMBATORE

PROJECT REPORT

ON

HANDWRITTEN DIGIT RECOGNITION USING CNN

Deep Learning Project

Submitted by

S. No.	Name	Register Number
1	Darsha Padmanaban	24BAD017
2	Jessica Sam. B	24BAD049
3	Dhanya G	24BAD019
4	Harshitha S K	24BAD034

Department of Artificial Intelligence and Data Science

Course: 24ADI306 – Deep Learning

Academic Year: 2026

ABSTRACT

Handwritten digit recognition is a computer vision task that identifies numerical digits from images of handwriting. This project presents a Convolutional Neural Network (CNN) designed to classify handwritten images into one of ten classes, representing the digits 0 through 9. The project uses the MNIST dataset, which contains 60,000 training images and 10,000 test images. Each image is a 28 × 28 grayscale image.

The workflow consists of loading the dataset, normalising pixel values, reshaping the images for CNN input, preparing labels, constructing the neural network, training the model, evaluating its performance, and using the trained model to predict a digit from a new image. The CNN architecture described in the project includes convolutional layers with 32 and 64 filters, max-pooling layers, a flattening stage, dropout, and a 10-unit softmax output layer. The presentation reports approximately 99% test accuracy; the final measured accuracy should be confirmed from the executed notebook before formal submission.

The project demonstrates how deep learning can automate the reading of handwritten digits and provides a foundation for applications such as cheque processing, postal PIN sorting, form digitisation, and optical character recognition (OCR).

Keywords: Deep Learning, Convolutional Neural Network, CNN, MNIST, Image Classification, Handwritten Digit Recognition.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Objectives
3. Literature Review
4. Existing and Proposed System
5. Dataset Description
6. System Requirements
7. Methodology and System Design
8. CNN Architecture
9. Implementation
10. Testing and Evaluation
11. Results and Discussion
12. Applications
13. Limitations and Future Scope
14. Conclusion
References
Appendix A – Core Implementation Code

1. INTRODUCTION

Handwritten digit recognition is the process of automatically identifying numerical digits written by people. Although digits appear simple, handwriting differs from person to person in shape, size, thickness, slant, spacing, and writing style. These variations make handwritten digit recognition a challenging image classification task.

Traditional rule-based approaches attempt to identify digits using manually defined features or conditions. Such rules may not work reliably when a digit is written in an unusual style. Deep learning offers an alternative by learning useful features directly from training images. A Convolutional Neural Network is particularly suitable for image data because it learns local patterns such as edges, curves, and strokes.

This project builds a CNN using Python and TensorFlow/Keras. The model is trained on the MNIST dataset and is intended to predict one of ten digit classes. The project also proposes a simple demonstration interface in which a user can draw or upload a digit and view the predicted class.

1.1 Background

Image classification assigns an input image to a category. In this project, the categories are the ten digits from 0 to 9. The network learns from labelled examples, where each training image is paired with its correct digit. During training, the model adjusts its parameters to reduce the difference between its predictions and the true labels.

1.2 Scope

The project focuses on single-digit classification using the MNIST dataset. It covers data preparation, CNN model construction, training, evaluation, and prediction. Recognition of handwritten letters, complete multi-digit strings, or unrestricted real-world documents is outside the core scope of the current version.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Handwritten digits vary in appearance, and simple rule-based techniques may fail to recognise them consistently. The problem addressed by this project is to develop a CNN that receives an image of a handwritten digit and predicts the correct numerical class from 0 to 9.

2.2 Objectives

- Load and use the MNIST dataset for supervised image classification.
- Preprocess the images by normalising pixel values and reshaping them to the required input format.
- Design and train a CNN using TensorFlow/Keras.
- Evaluate the trained model using accuracy, loss curves, and a confusion matrix.
- Provide a demonstration workflow for drawing or uploading a digit and viewing the prediction.
- Explore the relevance of digit recognition to digitisation and OCR-related tasks.

2.3 Expected Outcome

The intended outcome is a trained model that classifies MNIST test images with high accuracy and can generate predictions for new digit images. The presentation states a target/result of approximately 99% test accuracy. This value should be replaced with the exact measured score from the final run if available.

3. LITERATURE REVIEW

Handwritten digit recognition has been studied extensively in pattern recognition and computer vision. Early approaches commonly relied on manually engineered features, such as stroke direction, edges, and geometric properties, followed by a conventional classifier. Their performance could be affected by variations in writing style and image quality.

Machine learning methods improved classification by learning decision boundaries from labelled examples. However, image features often needed to be designed or selected separately. Deep learning methods reduce this dependency by learning representations from data.

Convolutional Neural Networks use learnable filters to identify local image patterns. Early convolutional layers can learn simple features, while deeper layers combine these patterns into more complex representations. Pooling reduces spatial dimensions and can help make learned features less sensitive to small shifts. CNNs are therefore a natural choice for handwritten digit classification.

The MNIST dataset is widely used as a benchmark for handwritten digit recognition because it provides labelled grayscale images in a consistent 28 × 28 format. In this project, MNIST provides the training and test examples used to develop and evaluate the CNN.

This review is a concise background summary. A formal literature review can be expanded with institution-approved research papers if specific citation requirements are provided.

4. EXISTING AND PROPOSED SYSTEM

4.1 Existing System

A basic existing approach may use manually defined rules or hand-engineered image features. Such methods can be sensitive to differences in stroke thickness, digit slant, and writing style. They may require additional adjustments when the input appearance changes.

4.2 Proposed System

The proposed system uses a CNN trained on MNIST. The network learns visual features from labelled examples and maps an input image to one of ten output probabilities. The class with the highest predicted probability is selected as the output digit.

Aspect	**Rule-based / feature-engineered approach**	**Proposed CNN approach**
Feature extraction	Often manually designed	Learned from training images
Handling visual variation	May require manual tuning	Learns patterns from diverse examples
Input in this project	Digit image	28 × 28 grayscale digit image
Output	Rule or classifier result	Probability distribution over digits 0–9
Evaluation	Depends on chosen method	Accuracy, loss, and confusion matrix

4.3 Advantages

- Learns image features automatically during training.
- Uses a standard labelled dataset for reproducible evaluation.
- Produces a probability score for each digit class.

- Can be extended to a simple interactive prediction interface.

5. DATASET DESCRIPTION

The project uses the MNIST handwritten digit dataset, loaded through keras.datasets.mnist. The dataset contains grayscale images of handwritten digits and their corresponding labels.

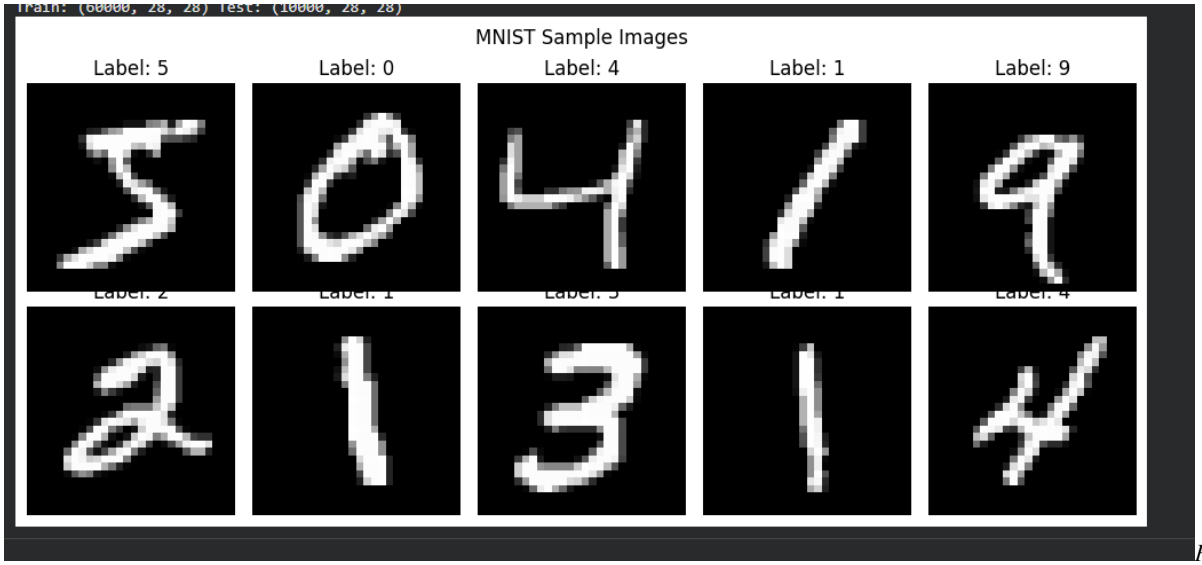


Figure 1. Representative sample images from the MNIST dataset.

Dataset property	Description
Training images	60,000
Test images	10,000
Image dimensions	28 × 28 pixels
Image type	Grayscale
Number of classes	10 (digits 0–9)
Label	Correct digit corresponding to each image

5.1 Data Preprocessing

Preprocessing prepares the raw images and labels for the neural network.

1. Load the MNIST training and test splits.
2. Convert pixel values from the 0–255 range to floating-point values in the 0–1 range by dividing by 255.
3. Reshape each image to include a single channel, producing the input shape (28, 28, 1).
4. Convert labels to one-hot encoded vectors when using categorical cross-entropy loss.

Normalisation places pixel values on a consistent scale, which helps neural-network optimisation. The channel dimension identifies the images as single-channel grayscale inputs. One-hot encoding represents each digit label as a vector of length ten.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- A computer capable of accessing Google Colab or Jupyter Notebook.

- A stable internet connection when using a cloud notebook.
- A GPU may be used to speed up training, but the MNIST model is small enough for a basic demonstration on many CPU environments.

6.2 Software Requirements

Software / Library	Purpose
Python	Programming language
TensorFlow / Keras	Dataset loading and CNN construction, training, and prediction
NumPy	Numerical array operations and preprocessing
Matplotlib	Visualising training curves and sample images
Google Colab / Jupyter	Notebook execution environment
Gradio or Streamlit	Optional user interface for drawing or uploading a digit
GitHub	Optional version control and project sharing

The PPT lists Gradio or Streamlit as demo options. The report does not assume that either interface has already been implemented; the chosen option should match the actual project demonstration.

7. METHODOLOGY AND SYSTEM DESIGN

The project follows a sequential machine-learning pipeline: load the dataset, preprocess the images and labels, build the CNN, train the model, evaluate its performance, and use the trained model for prediction.

Stage	Main activity	Output
1. Load dataset	Load MNIST training and test data	Images and labels
2. Preprocess	Normalise, reshape, and encode labels	Model-ready data
3. Build CNN	Define convolution, pooling, flatten, dropout, and output layers	Compiled model structure
4. Train	Fit the model to labelled training images	Learned model parameters
5. Evaluate	Calculate test metrics and inspect errors	Accuracy, loss, confusion matrix
6. Predict	Prepare a new image and run inference	Predicted digit and class probabilities

7.1 Convolutional Layer

A convolutional layer applies learnable filters to an input image. The filters respond to local patterns such as edges, curves, and strokes. The ReLU activation function introduces non-linearity by replacing negative activations with zero.

7.2 Max-Pooling Layer

Max pooling reduces the spatial dimensions of feature maps by selecting the maximum value within each pooling window. This reduces computation and retains strong local activations.

7.3 Flatten Layer

The flatten operation converts the multi-dimensional feature maps into a one-dimensional vector so that the extracted features can be passed to the output classifier.

7.4 Dropout

Dropout randomly disables a proportion of activations during training. The project specifies a dropout rate of 0.5, which is intended to reduce overfitting and improve generalisation.

7.5 Softmax Output

The final dense layer contains ten units, one for each digit class. Softmax converts the output scores into a probability distribution. The predicted digit is the class with the highest probability.

8. CNN ARCHITECTURE

Layer	Configuration in the PPT	Purpose
Convolution 1	32 filters, 3 × 3 kernel, ReLU	Extract basic edges and strokes
Max Pooling 1	2 × 2 pool	Reduce spatial size
Convolution 2	64 filters, 3 × 3 kernel, ReLU	Learn more complex shapes
Max Pooling 2	2 × 2 pool	Further down-sampling
Flatten	Convert feature maps to a vector	Prepare features for classification
Dropout	Rate 0.5	Help reduce overfitting
Dense output	10 units, Softmax	Predict probabilities for digits 0–9

The exact input and intermediate tensor dimensions depend on layer settings such as padding and stride. The architecture table above reproduces the configuration described in the presentation. The final executable model should be checked against the actual notebook implementation.

```
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()   # SCREENSHOT 2: model summary output
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)

Figure 2. Model summary generated by the executed Colab notebook.

9. IMPLEMENTATION

9.1 Development Environment

The project can be implemented in Google Colab using Python, TensorFlow/Keras, NumPy, and Matplotlib. The MNIST dataset can be loaded directly through the Keras dataset utility, so a separate manual dataset download is not required for the basic notebook.

9.2 Training Configuration

Parameter	Configuration stated in the PPT
Optimiser	Adam
Epochs	10
Batch size	128
Loss function	Sparse categorical cross-entropy
Dataset	MNIST
Output classes	10

9.3 Training Process

5. Load the training and test images and labels.
6. Normalise the images and reshape them to the CNN input format.
7. Convert labels to one-hot encoded vectors if using categorical cross-entropy.
8. Construct the CNN using the selected convolution, pooling, flatten, dropout, and dense layers.
9. Compile the model with the Adam optimiser, categorical cross-entropy loss, and accuracy metric.
10. Train for the configured number of epochs and record training and validation metrics.
11. Evaluate the final model on the test dataset and record the exact results.
12. Use the trained model to predict a digit from a correctly preprocessed input image.

9.4 Prediction Workflow

For a new digit image, the image should be converted to grayscale, resized to 28 × 28 pixels, and normalised consistently with the training data. The image should also be oriented and inverted if necessary so its background and foreground match the MNIST convention. The processed image is then reshaped to include the batch and channel dimensions before calling the model's prediction method.

If a drawing or upload interface is used, the interface should display the predicted digit and may also display the probability associated with the predicted class. Input preprocessing is important: a model trained on centred, light digits on a dark background may perform poorly if the uploaded image uses the opposite convention.

10. TESTING AND EVALUATION

10.1 Evaluation Metrics

Accuracy measures the fraction of examples classified correctly. It is calculated as the number of correct predictions divided by the total number of predictions.

Loss measures the discrepancy between the predicted class probabilities and the true labels. Categorical cross-entropy is the loss function specified in the presentation.

A confusion matrix compares actual classes with predicted classes. It helps identify which digits are confused with other digits. For example, similar-looking handwritten digits may be misclassified when their strokes are ambiguous.

10.2 Test Cases

Test case	Input / action	Expected behaviour
Dataset loading	Load MNIST through Keras	Training and test splits load successfully
Preprocessing	Normalise and reshape images	Pixel range and tensor dimensions match the model input
Model training	Train with configured settings	Training completes and metrics are recorded
Test evaluation	Evaluate on held-out test images	Test loss and accuracy are returned
Known sample	Predict a test image	Predicted digit can be compared with its label
New image	Draw or upload a digit	Image is preprocessed and a digit prediction is displayed

10.3 Validation Notes

The training and test sets should remain separate. Test results should be computed on the held-out test set rather than reused as training feedback. If a validation split is used during training, its method and size should be recorded in the final report. The presentation mentions accuracy and loss curves and a confusion matrix but does not contain the actual plots; screenshots should be inserted after they are generated from the notebook.

11. RESULTS AND DISCUSSION

11.1 Results Reported in the Presentation

The presentation reports approximately 99% test accuracy, with Adam as the optimiser, 10 epochs, batch size 128, and categorical cross-entropy as the loss function. It also states that training and validation accuracy rise together, suggesting limited overfitting. The presentation contains placeholders for accuracy/loss graphs and a confusion matrix rather than the actual figures.

Measure	Value / status
Test accuracy	99.09% (reported by the uploaded Colab notebook)
Optimiser	Adam
Epochs	10
Batch size	128
Loss function	Sparse categorical cross-entropy
Accuracy and loss curves	Insert screenshots from the executed notebook
Confusion matrix	Insert the generated matrix from the executed notebook

11.2 Discussion

A high test accuracy would indicate that the CNN has learned patterns that generalise well to unseen MNIST images. Convolutional filters help capture local stroke patterns, while pooling reduces feature-map dimensions. Dropout is included to help control overfitting. However, the exact performance depends on the implementation, random initialisation, preprocessing, and training run. Therefore, the final submission should report the measured test accuracy and include actual plots generated by the code.

11.3 Figures to Add

Training and validation accuracy curve included above.

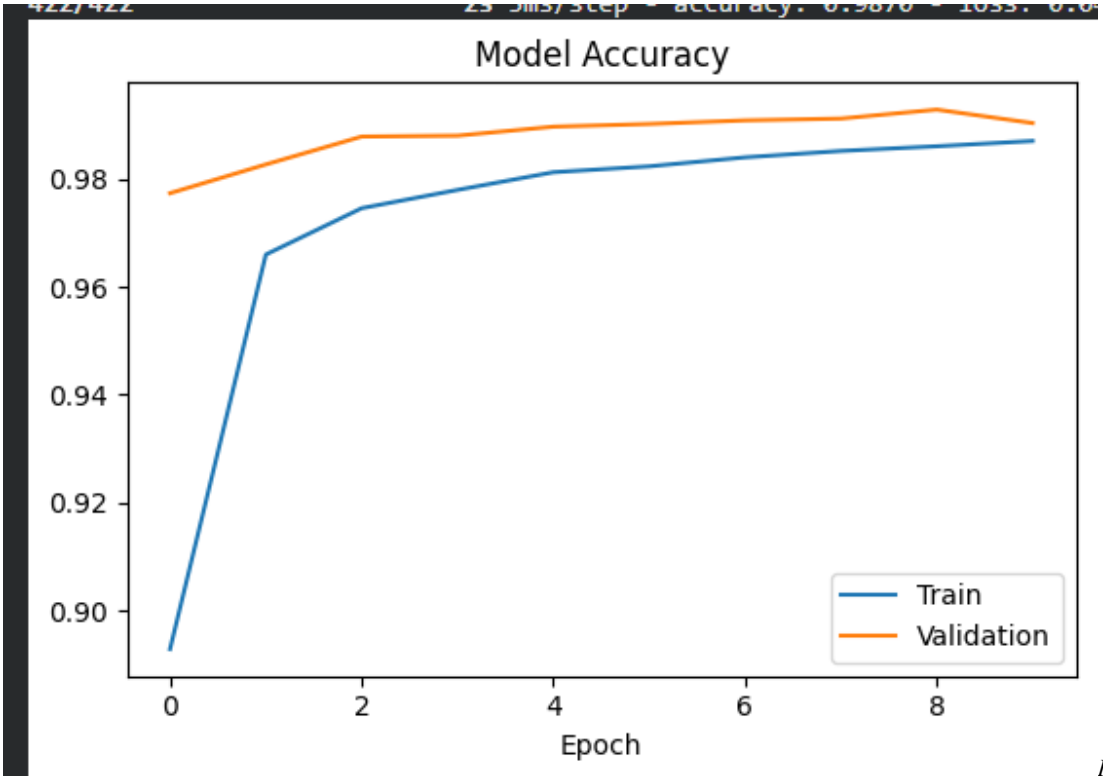


Figure 3. Training and validation accuracy across 10 epochs.

Training and validation loss curve included above.

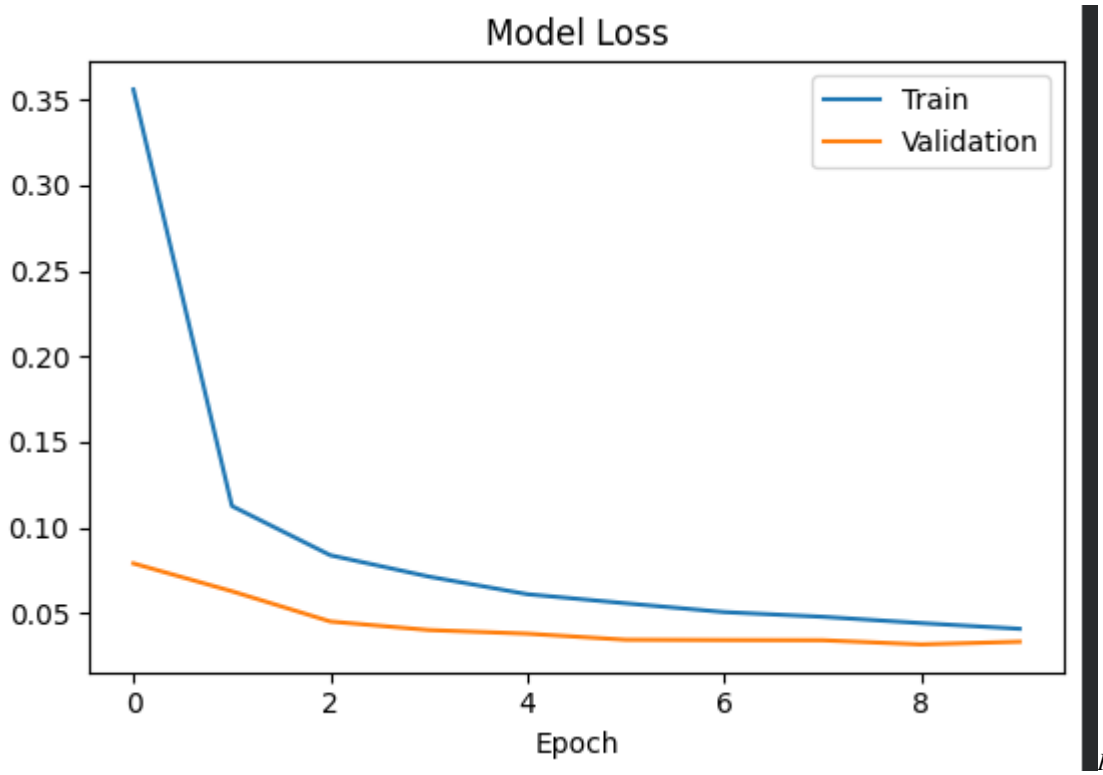


Figure 4. Training and validation loss across 10 epochs.

Confusion matrix included above.

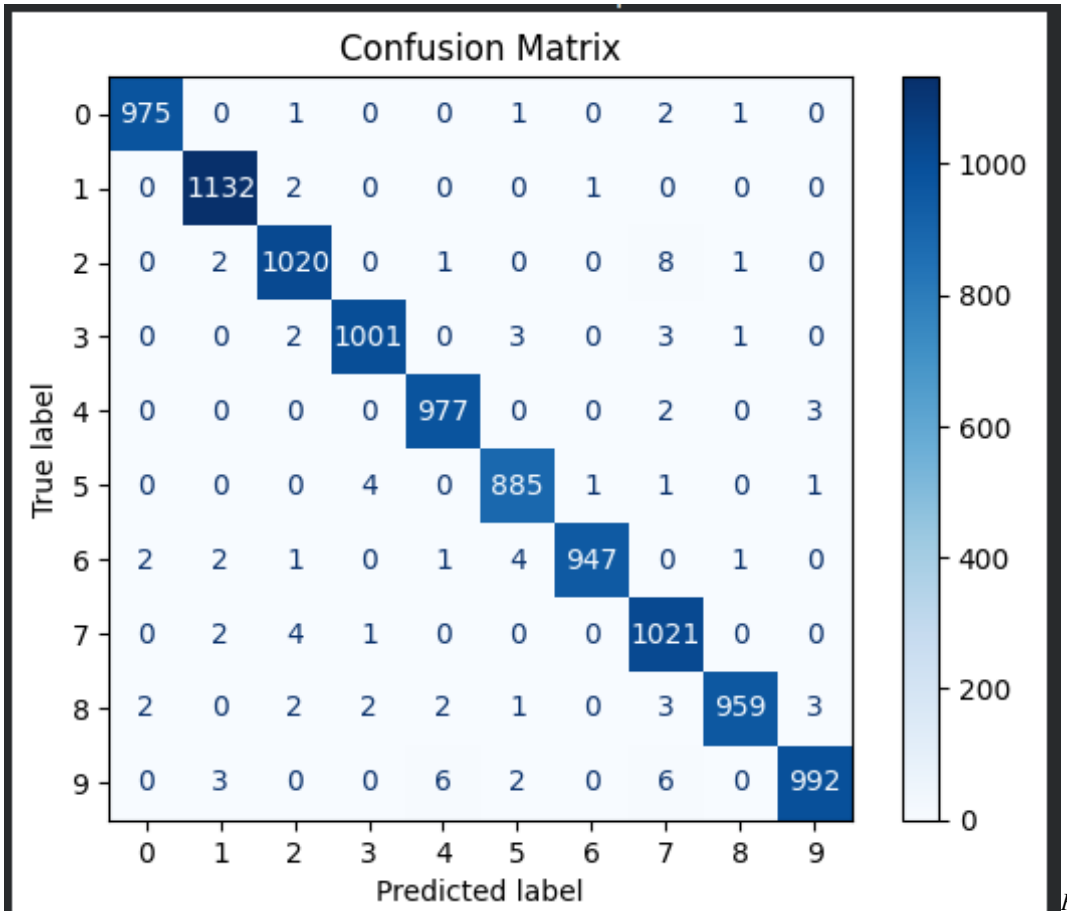


Figure 5. Confusion matrix for digit classes 0–9.

Sample prediction outputs included above.

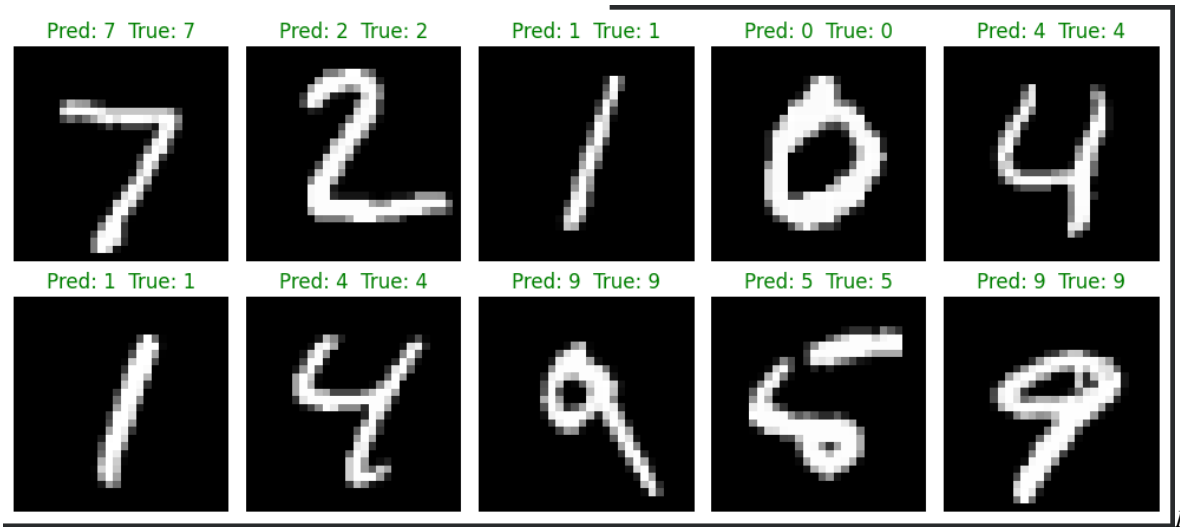


Figure 6. Sample predictions compared with the true labels.

12. APPLICATIONS

- Bank cheque processing: recognising handwritten numerical amounts or fields as part of a larger document-processing workflow.

- Postal PIN sorting: helping read handwritten postal codes when integrated with a complete postal recognition system.
- Form digitisation: converting handwritten numerical fields into digital data.
- Examination and survey processing: supporting digit recognition in forms, subject to appropriate accuracy checks.
- Number plate and meter reading: serving as a foundational technique for recognising individual digits within a larger computer-vision system.
- Optical character recognition: acting as a starting point for more comprehensive systems that recognise letters, words, and multi-digit sequences.

These are potential application areas. The current project demonstrates single-digit classification on MNIST and does not, by itself, implement a complete cheque, postal, number-plate, or document-processing system.

13. LIMITATIONS AND FUTURE SCOPE

13.1 Limitations

- The current task focuses on isolated digits rather than complete words or multi-digit strings.
- MNIST images are small, grayscale, and relatively consistent compared with photographs or scanned documents from real-world environments.
- Prediction quality can be affected by image alignment, background colour, contrast, stroke thickness, and preprocessing.
- The reported accuracy must be confirmed using the actual test output.
- A draw/upload interface requires additional implementation and testing beyond model training.

13.2 Future Scope

- Extend the model to handwritten letters and alphanumeric characters.
- Develop recognition for multi-digit numbers and complete handwritten fields.
- Use data augmentation to improve robustness to shifts, rotations, and variations in writing style.
- Improve input preprocessing for images uploaded from phones or scanned documents.
- Deploy the model through a web or mobile interface.
- Integrate the digit classifier into a larger OCR pipeline with text segmentation and post-processing.

14. CONCLUSION

This project presents a CNN-based approach for handwritten digit recognition using the MNIST dataset. The workflow covers dataset loading, image normalisation, reshaping, CNN construction, model training, evaluation, and prediction. The network described in the presentation uses convolutional layers with 32 and 64 filters, max pooling, flattening, dropout, and a ten-class softmax output.

The presentation reports approximately 99% test accuracy and identifies applications such as cheque processing, postal code sorting, form digitisation, and OCR. The model provides a practical introduction to deep learning for image classification. Before final submission, the reported accuracy should be checked against the actual notebook output, and the accuracy/loss plots, confusion matrix, and demo screenshot should be inserted if available. Future work can extend the system to letters, multi-digit numbers, data augmentation, and deployment.

REFERENCES

13. LeCun, Y., Cortes, C., and Burges, C. J. C. The MNIST Database of Handwritten Digits. http://yann.lecun.com/exdb/mnist/
14. TensorFlow. Keras API documentation. https://www.tensorflow.org/api_docs/python/tf/keras
15. TensorFlow. MNIST handwritten digits classification tutorial and dataset utilities. https://www.tensorflow.org/
16. Goodfellow, I., Bengio, Y., and Courville, A. Deep Learning. MIT Press, 2016. https://www.deeplearningbook.org/
17. Project presentation: Handwritten Digit Recognition using CNN, Team 8, Deep Learning Project.

APPENDIX A – CORE IMPLEMENTATION CODE

The following example demonstrates the core MNIST CNN workflow described in the project presentation. Run it in Google Colab or a compatible Python environment. If the team's actual notebook differs, the submitted code and report should be aligned before submission.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Labels remain integer encoded for sparse categorical cross-entropy


model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1
```

```
)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print("Test loss:", test_loss)
print("Test accuracy:", test_accuracy)


plt.figure()
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()


plt.figure()
plt.plot(history.history["loss"], label="Training loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

predictions = model.predict(x_test[:1], verbose=0)
print("Predicted digit:", int(np.argmax(predictions[0])))
print("Actual digit:", int(y_test[0]))
```